

임장혁 자기소개서

Backend Developer · qkdrhkgn23@naver.com · github.com/imjanghyeok

지원 직무와 제 강점

저는 백엔드 개발자로서 상태가 변하는 시스템을 안정적으로 다루는 데 관심이 있습니다. 지금까지 수행한 프로젝트는 AI 면접 서비스의 세션 흐름, MSA 메신저의 메시지 전파와 히스토리 조회, WebSocket 채팅의 접속 상태와 미수신 메시지 복구처럼 단순 CRUD보다 상태 변화와 비동기 흐름이 중요한 문제들이었습니다. Java/Spring Boot와 TypeScript/NestJS를 사용했고, Redis, Kafka, WebSocket/STOMP, Docker, GitHub Actions를 프로젝트 안에서 직접 다뤘습니다.

문제를 기능이 아닌 시스템 관점에서 보는 이유

저는 기능이 동작하는지보다, 실패했을 때 사용자가 무엇을 잃는지를 먼저 확인하려고 합니다. 이 관점은 WebSocket 채팅, Kafka 기반 메시지 전파, LLM 기반 면접 흐름 제어를 구현하면서 더 명확해졌습니다. 채팅에서는 메시지가 실제로 유실되지 않아도 조회 시점이 늦으면 사용자는 유실로 받아들일 수 있었고, LLM 면접에서는 질문이 반복되거나 종료가 어색하면 사용자는 서비스가 답변을 이해하지 못한다고 느낄 수 있었습니다. 그래서 저는 기능 구현을 끝으로 보지 않고, 상태가 어디에 저장되는지, 흐름이 어디에서 결정되는지, 실패를 어디에서 멈추고 복구할 수 있는지까지 확인하는 개발자를 지향합니다.

검증 가능한 구조를 만들기 위한 경험

Synapse 프로젝트에서는 AI 면접 흐름을 다루며 LLM 출력을 그대로 신뢰하지 않고, 서버가 종료 조건과 중복 방지 정책을 다시 판단하도록 분리했습니다. 면접 중 필요한 질문 이력과 방문 주제는 영속 데이터가 아니라 만료되어야 하는 런타임 상태로 보았고, `KeySetStore`와 `MinHeapScheduler`를 통해 TTL 기반 정리 흐름을 구현했습니다. 또한 문서 일괄 삭제에서는 연관 데이터 삭제를 애플리케이션 반복 로직에 맡기기보다 DB의 `ON DELETE CASCADE` 제약으로 위임하고, 요청 DTO 검증과 E2E 테스트를 함께 추가했습니다.

이 과정에서 제가 중요하게 본 것은 “더 멋진 구조”가 아니라 “검증할 수 있는 구조”였습니다. CI 파이프라인을 Lint, Build, Unit, E2E 단계로 나누고, Auth/User/Document 모듈의 E2E 테스트와 `MinHeapScheduler`, `KeySetStore` 단위 테스트를 보강했습니다. 구현한 기능이 왜 필요한지, 어떤 실패를 막으려는지, 어떤 테스트로 확인했는지를 문서와 코드 리뷰에 남기는 방식으로 팀원이 같은 판단 기준을 공유할 수 있게 했습니다.

실시간 서비스에서 배운 책임 분리

FaceFriend 프로젝트에서는 WebSocket/STOMP 기반 채팅을 구현하며 접속 상태와 채팅방 입장 상태를 Redis에 분리해 저장했습니다. 메시지 자체는 영속 데이터로 관리하되, 사용자가 지금 접속 중인지와 같은 현재성 정보는 빠르게 변하는 런타임 상태로 보았습니다. 또한 STOMP 통신 중 오류가 발생했을 때 프론트엔드가 실패를 인지할 수 있도록 오류 응답 흐름을 정리했습니다. 이 경험을 통해 실시간 기능에서는 “저장된 데이터”와 “현재 상태”를 같은 방식으로 다루면 안 된다는 점을 배웠습니다.

비동기 시스템에서 사용자가 체감하는 정합성을 다룬 경험

SmileTogether 프로젝트에서는 채팅 서버, 히스토리 서버, 알림 서버가 분리된 구조에서 메시지 전파와 조회 정합성을 다뤘습니다. 채팅 메시지가 Kafka를 통해 비동기로 저장되기 때문에, 사용자가 전송 직후 히스토리를 조회하면 아직 저장되지 않은 메시지가 누락처럼 보일 수 있습니다. 이를 완화하기 위해 `KafkaLagChecker`를 추가해 latest offset과 committed offset을 비교하고, lag이 남아 있으면 최대 5회, 100ms 간격으로 polling한 뒤 MongoDB를 조회하도록 변경했습니다.

또한 알림 서버 MVP를 별도로 구성해 채팅 서버가 알림 책임까지 가지지 않도록 분리했습니다. FCM 구독 정보를 저장하는 API, Kafka consumer, FCM 전송 DTO와 서비스 흐름을 만들고, 이후 서버 간 통신과 CORS 문제를 해결하며 하드코딩된 대상이 아니라 채널 멤버를 기준

으로 알림을 보낼 수 있게 수정했습니다. 이 경험을 통해 실시간 서비스의 신뢰성은 단일 API 성공 여부가 아니라 전파, 저장, 조회, 알림이 각자의 책임 안에서 연결될 때 만들어진다는 점을 배웠습니다.

협업과 학습 방식

저는 코드 작성뿐 아니라 협업 방식 자체도 개발 품질의 일부라고 생각합니다. 네이버 부스트캠프 웹·모바일 10기에서는 매일 제한된 시간 안에 문제를 분석하고 구현하며, 동료와 리뷰를 주고받는 방식으로 개발했습니다. 우아한테크코스 프리코스에서는 문자열 계산기, 자동차 경주, 로또 미션을 구현하며 MVC, Validator 분리, Strategy 패턴을 적용했고, 동료 코드 리뷰에서 DTO 책임, 인터페이스 활용, 네이밍에 대해 의견을 나눴습니다. Spring Session 공식 문서에서는 MongoDB와 Hazelcast 커뮤니티 익스텐션 항목이 누락된 것을 확인하고 수정 제안을 제출했습니다.

이러한 경험들은 제가 빠르게 구현하는 사람보다, 문제의 맥락을 확인하고 동료가 검토할 수 있는 형태로 남기는 개발자가 되어야 한다는 기준을 만들었습니다. 앞으로도 확인되지 않은 성과를 크게 말하기보다, 제가 직접 구현하고 검증한 사실을 바탕으로 팀이 신뢰할 수 있는 백엔드 시스템을 만드는 데 기여하고 싶습니다.

입사 후 기여하고 싶은 방식

저는 새로운 기술을 빠르게 나열하기보다, 맡은 도메인에서 실패 조건을 먼저 정의하고 그 조건을 코드와 테스트에 반영하는 방식으로 기여하고 싶습니다. 특히 백엔드 시스템에서는 데이터 정합성, 비동기 처리 지연, 세션 상태 만료, 배포 환경 차이가 사용자 경험으로 바로 드러난다고 생각합니다. 제가 해온 프로젝트들은 모두 이 문제를 작게나마 다룬 경험이었고, 앞으로도 확인 가능한 사실과 실행 가능한 개선을 기준으로 팀에 기여하겠습니다.